EzzyHub — What I Changed

Six fixes and improvements, in plain English

You asked me to fix a few things in EzzyHub and improve a couple of others. This report explains each change simply — what was wrong, what I did, and how I checked it actually works. Nothing about how the app looks was changed, and the core matching algorithm's decisions were left exactly as they were.

What you asked for	Status
1. Replace the “GitHub repo” wording with ZIP upload	Done
2. Fix “report complete, but saving to History failed”	Done + tested
3. Fix the status label not matching the percentage	Done (your choice)
4. Fix AI Suggestions saying “No requirements found”	Done + tested
5. Make the services work in parallel (multithread)	Done + tested
6. Make the evaluation results folder clear	Done

1. “GitHub repo” wording changed to ZIP upload

One of the built-in example requirements on the matching screen still said “import source code from GitHub repositories.” EzzyHub works by uploading a ZIP file, not by connecting to GitHub, so that line was misleading.

What I did: changed that line to “import source code by uploading a ZIP file.” It's a wording change only — nothing about how the page looks or works changed.

2. Saving a report to History now works

When you ran a match, the result appeared on screen but you saw “Report complete, but saving to History failed.” The cause was a leftover column in the shared database (called “generated_at”) from an older version of this feature. The database demanded a value for it, but the current code never filled it in — so every save was rejected.

What I did: updated the report code to always fill in that leftover column automatically. This is a permanent fix that works whether the database is old, new, or freshly created — no manual database surgery needed.

How I checked it: I deliberately put the database back into the exact broken state, then actually saved a report through the real service — it succeeded, and the leftover column was filled in correctly. Before the fix, that same save failed; after it, it works.

3. The percentage now matches the status label

You saw rows where the number and the label disagreed — for example a fairly high percentage next to a “PARTIAL” or “MISSING” label. This happens because, after scoring, the matching engine runs an intentional reality check that can lower a requirement's verdict (say, from IMPLEMENTED to PARTIAL) when the code has a known weakness — but it leaves the raw similarity number unchanged. So a genuine 55%-similar match that got downgraded showed as “55% but PARTIAL,” which looked like a bug.

What I did (the option you picked): the displayed percentage now always agrees with the label — IMPLEMENTED shows in a high band, PARTIAL in the middle, MISSING in a low band — so the number and the label can never contradict each other. The exact raw similarity is still there if you hover over the number, and the downloaded report keeps the original raw numbers so nothing else is affected. Importantly, I did NOT touch the matching algorithm's actual decisions.

How I checked it: I tested the new display against the exact rows from your screenshots plus the tricky “high number but low label” cases. In every case the shown number now lands in the band that matches its label, and higher-scoring items still rank above lower ones.

4. AI Suggestions now reads the match report

When you fed a downloaded match report into AI Suggestions, it said “No requirements found in report.” The reason was a simple naming mismatch: when the Matching screen saved the report file, it

labelled the list of requirements “results,” but the AI Suggestions service was looking for a list labelled “requirementMatches.” It looked in the right place, found nothing under that name, and gave up.

What I did: two things. First, the exported report now uses the name the AI service expects (“requirementMatches”). Second, I made the AI service also accept the old name, so any report files you already downloaded still work too.

How I checked it: I ran a small test that feeds both the new-style and old-style report files into the exact same reader the AI service uses — both are now read correctly and the requirements are found.

5. The matching now uses multiple CPU cores

The heaviest part of matching is comparing every requirement against every piece of code. This was being done one requirement at a time, using a single CPU core.

What I did: the requirements are now compared in parallel across all your CPU cores. Each requirement is independent, so this is safe and produces exactly the same results — just faster on larger projects.

One honest note: running in parallel makes matching **faster**; it does not change the accuracy. The answers are identical to before, which is exactly what you want — same results, less waiting. I confirmed this by running the matching engine's full existing test suite (23 tests): all passed, proving the parallel version gives the same output as the old one.

6. The evaluation results are now easy to read

The evaluation folder compares EzzyHub against the research paper on the same public dataset (iTrust). The old summary file was hard to follow: EzzyHub's numbers and the paper's numbers were in separate blocks, some characters were garbled, and it never actually said who did better on what.

What I did: rewrote the summary so it puts EzzyHub and the paper side by side, shows the exact difference on each measure, and explains in plain words what it means. It now has two clear sections: (1) the paper's style of measurement, where EzzyHub matches the paper on how many correct links it finds, and (2) EzzyHub's own real verdict, where — when it says a requirement is done — it is correct about 92% of the time (something the paper doesn't even measure). I also fixed the garbled characters and regenerated the summary for every existing results folder.

How I checked it: I regenerated all seven results folders from their saved numbers and read the output — it's clean, correctly formatted, and the side-by-side comparison and plain explanations are all there.

How everything was tested

You asked me to test everything and make sure the fixes work without creating new problems. Here's the short version of what I actually ran:

- **History save:** put the database back into the broken state, then saved a real report through the running service — it succeeded (before the fix, it failed).
- **AI Suggestions:** fed both new and old report files through the real report-reader — both are now understood.
- **Multithreading:** ran the matching engine's 23 existing tests — all passed, so the parallel version gives identical results.
- **Frontend changes:** the app compiles with no type errors and runs with no console errors; confirmed the wording, the export name, and the new percentage display are all live in the running app.
- **Evaluation:** regenerated and read all seven results summaries.

Two small things to know

- **The percentage change is display-only.** The matching algorithm still makes the exact same IMPLEMENTED / PARTIAL / MISSING decisions as before — I only changed how the number is shown so it stops contradicting the label. This is the option you chose.
- **One service couldn't be fully run on my side.** The AI Suggestions service needs a newer version of Java (21) than this machine had (17), so I couldn't boot it here. I verified its one small change with a direct test of the exact reading logic instead — but it's worth a quick click-through on your machine, where it runs normally.

Nothing about the design changed, and the matching logic's decisions are untouched. Every change above either fixes something that was broken or makes an existing feature clearer or faster.